

Code Review Agent

Project Design Document

Vaishnavi V Chaughule | MSCS, Northeastern University Seattle | March 2026

What Is This Project?

The Code Review Agent is a personalized AI-powered code review system. Unlike generic tools such as GitHub Copilot that apply universal internet rules, this agent learns YOUR specific coding patterns from your own GitHub repository and uses them as the standard for reviewing any new code you write.

You paste a GitHub repo URL. The agent reads your entire codebase, builds a fingerprint of how you specifically write code — your naming style, function structure, error handling, docstring format, and more. Then when you submit new code for review, it compares it against YOUR patterns, not generic rules. It gives you line-by-line feedback with severity scores and exact fixes.

What Is Achieved?

Personalized Reviews: Feedback based on YOUR codebase, not generic internet standards.

Pattern Fingerprinting: Extracts 7 coding patterns per language to build a unique developer profile.

Structured Output: Line-by-line issues with severity (Critical / Warning / Suggestion), score out of 100, side-by-side diffs, and comparison stats.

Live Streaming: Review output streams word by word like ChatGPT — impressive in demos.

Multi-language Support: Analyzes Python, Java, JavaScript, Kotlin, TypeScript, and Shell files.

Downloadable Reports: Export full review as PDF or JSON for documentation and sharing.

Smart Caching: Saves analyzed repo fingerprints locally — no re-analyzing the same repo twice.

Technology Stack

Component	Technology	Purpose
LLM — Review Generation	Groq (Llama 3.3 70B)	Generates personalized code review feedback
LLM — Embeddings	Gemini (text-embedding-004)	Converts code chunks to vectors
Vector Store	FAISS (local)	Stores and retrieves similar code patterns
Repo Ingestion	PyGithub	Reads files from any GitHub repo
Data Validation	Pydantic v2	Enforces structured review output schemas
UI & Deployment	Streamlit	Interactive web app with live streaming

Secret Management	.env + python-dotenv	Securely stores API keys
CI/CD	GitHub Actions	Automated testing and deployment

Who Is This Built For?

Individual Developers: Who want personalized code reviews based on their own style rather than generic linters.

Engineering Teams: Who have internal conventions that no existing tool enforces — this agent learns those conventions automatically.

CS Students & Bootcamp Graduates: Who want to improve code quality and consistency across their portfolio projects.

Tech Interviewers & Recruiters: Who can see a live demo of AI-powered personalized code intelligence — not another to-do app.

Open Source Maintainers: Who want to enforce contributor coding standards automatically on PRs.

Design Decisions

The following decisions were made by Vaishnavi during the design phase before any code was written. Each decision directly shaped the project architecture and file structure.

Decision 1 — Repository Ingestion

Repo Type Support:

Both public and private repositories (PAT required for private repos)

File Types Analyzed:

.py (Python), .java (Java), .js (JavaScript), .kt (Kotlin), .ts (TypeScript), .sh (Shell/Bash)

Repo Scope:

Both options — analyze entire repo AND let user pick specific folders

Why This Decision:

Supporting all 6 file types makes the agent useful across your entire portfolio (Python ML projects, Java coursework, Kotlin Android apps). Folder selection gives users control for large repos.

Decision 2 — Pattern Extraction (The Fingerprint)

Patterns Extracted:

All 7 patterns selected: Naming conventions, Average function length, Docstring coverage & style, Error handling style, Import organization, Type hints usage, Comment density

Fingerprint Scope:

Both combined (across all languages) + per-language breakdown

Verification Step:

Agent shows the extracted fingerprint BEFORE running the review (so user can verify it is accurate)

Why This Decision:

Showing the fingerprint first builds user trust. Per-language breakdown is essential because Python and Java have completely different conventions.

Decision 3 — Review Output

Output Elements:

All 5 selected: Line-by-line issues with severity (Critical / Warning / Suggestion), Overall summary paragraph, Side-by-side diff (original vs suggested fix), Score out of 100, Comparison stats (your avg vs this code)

Code Submission:

Both — paste code directly into UI OR upload a file

Output Display:

Streams live word by word (like ChatGPT) — not shown all at once

Why This Decision:

Live streaming makes the demo dramatically more impressive in interviews. Score out of 100 gives a memorable, shareable number. Comparison stats like 'your avg function length is 18, this one is 47' are what make reviewers say wow.

Decision 4 — Storage and UI

Repo Caching:

Save analyzed repo fingerprints locally with option to force refresh (avoids re-analyzing the same repo every time)

Report Export:

Both PDF and JSON download options

UI Theme:

Light and dark mode with a toggle switch

Why This Decision:

Caching makes the app feel production-grade and fast. PDF + JSON export means the review can be shared, stored, or integrated into other workflows. Dark mode toggle is expected by developers.

Project File Structure

Each file below was created as a direct result of a design decision above:

File	Status	Purpose
models.py	DONE	Pydantic data schemas — foundation for all other files. Includes Severity enum, ReviewIssue, C
github_ingestor.py	NEXT	Pulls files from GitHub using PyGithub. Handles public/private repos, all 6 file types, entire repo
pattern_extractor.py	PENDING	Analyzes pulled files to extract all 7 coding patterns. Returns both combined and per-language F
embedder.py	PENDING	Converts code chunks to vectors using Gemini embeddings. Stores in FAISS locally for fast retri
cache_manager.py	PENDING	Saves and loads analyzed repo fingerprints locally. Created because of the 'save with force refr

reviewer.py	PENDING	Core review logic. Uses Groq to compare new code against fingerprint. Streams output. Returns
report_generator.py	PENDING	Exports review as PDF and JSON. Created because of the 'download report' design decision.
app.py	PENDING	Streamlit UI. Light/dark toggle, live streaming, paste or upload input, fingerprint verification step.

Build Order

Files are built in this exact order so that each file can import from the ones before it without errors:

1. models.py — data contract (foundation everything else imports)
2. github_ingestor.py — repo ingestion (needs models for file type config)
3. pattern_extractor.py — fingerprint extraction (needs models.RepoPattern)
4. embedder.py — vector store (needs pattern_extractor output)
5. cache_manager.py — local storage (needs embedder + pattern output)
6. reviewer.py — review engine (needs all above)
7. report_generator.py — export (needs reviewer output)
8. app.py — UI (depends on everything above)